

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ
Факультет комп'ютерних наук та технологій
Кафедра програмної інженерії

ЗВІТ
з лабораторної роботи № 4
«Рефакторинг коду та проведення Code Review»
з дисципліни «Основи програмної інженерії»

Виконав: студент групи ПЗПІ-25-4

Могила Ярослав Романович

Перевірив: Ст. Викладач кафедри програмної інженерії

В'ячеслав Гребенюк

Харків — 2026

1. Мета роботи

Мета роботи — навчитися робити код-рев'ю чужого коду та проводити рефакторинг на основі знайдених зауважень. У цій роботі аналізується код калькулятора витрат на по-
code проєкти з лабораторної роботи № 3. За результатами аналізу проводиться рефакторинг, після якого всі тести мають залишитись робочими.

Файли для аналізу: `calc.py` та `test_calculator.py`. Репозиторій: <https://github.com/yarikknure/bse-lr3-Mohyla>

2. Аналіз вихідного коду

2.1. Структура проєкту

Проєкт складається з двох файлів:

- `calc.py` (~190 рядків) — класи `User` і `ProjectCalculator`, функція `run_calculation`.
- `test_calculator.py` (411 рядків) — 39 тестів, написаних за методиками EP та BVA.

Код написаний охайно: є docstrings, type hints, витримано PEP 8. Перед рефакторингом усі 39 тестів проходять. Проте під час перегляду знайшлося кілька структурних проблем.

2.2. Знайдені проблеми

Таблиця 1 — Результати Code Review

№	Файл / рядки	Тип проблеми	Опис та рекомендація	Пріоритет
1	<code>calc.py</code> , рядки 61–160 (клас <code>ProjectCalculator</code>)	Клас з кількома обов'язками (SRP)	Клас <code>ProjectCalculator</code> одночасно зберігає тарифи в <code>__init__</code> , рахує вартість у <code>calculate_cost()</code> і формує звіт у <code>generate_report()</code> . Це три різні задачі в одному місці. Рекомендація: винести <code>calculate_cost()</code> і <code>generate_report()</code> у звичайні функції модуля, а тарифи зробити константами.	Високий
2	<code>calc.py</code> , рядки 38–39, 57–58	Дублювання стану	Є два поля, що описують одне й те саме: <code>calculation_history</code> (список) і <code>calculation_count</code> (лічильник). В методі <code>add_to_history()</code> вони синхронізуються вручну. Рекомендація: прибрати <code>calculation_count</code> і зробити його <code>@property</code> , що повертає <code>len(self.calculation_history)</code> .	Високий
3	<code>calc.py</code> , рядки 69–83	Магічні числа / розкидані константи	<code>COMPLEXITY_FACTORS</code> оголошено всередині класу, <code>platform_rates</code> зберігається як атрибут об'єкта в <code>__init__</code> . Мітки складності 'Низька'/'Середня'/'Висока' продубльовано окремо в <code>generate_report</code> . Рекомендація: зібрати всі тарифи і множники у	Середній

№	Файл / рядки	Тип проблеми	Опис та рекомендація	Пріоритет
			словники-константи на рівні модуля (PLATFORM_RATES, COMPLEXITY_LABELS).	
4	calc.py, рядки 88–105 (calculate_cost)	Зайвий цикл, неповна перевірка помилок	base_cost рахується через for-цикл з накопиченням. При невідомій платформі функція зупиняється одразу після першої помилки. Рекомендація: base_cost = sum(PLATFORM_RATES[p] for p in platforms); зібрати всі невідомі платформи одразу і показати їх разом в одному ValueError.	Середній
5	calc.py, рядки 110–160 (generate_report)	Пряме звернення до полів об'єкта	generate_report() напругу читає user.calculation_history і user.is_registered. Число 50 у виразі '=' * 50 вшиті без пояснення. Рекомендація: ввести константу REPORT_WIDTH = 50; вважати кращим передавати у функцію лише потрібні дані.	Середній
6	calc.py, рядки 162–190 (run_calculation)	Зайва залежність, змішані задачі	run_calculation() приймає об'єкт ProjectCalculator як параметр, хоча він там потрібний лише для одного виклику. Функція одночасно рахує вартість і виводить результат через print(). Рекомендація: зробити calculate_cost() звичайною функцією; run_calculation більше не потребуватиме calculator.	Середній
7	test_calculator.py, рядки 80, 88, 96, 104, 112, 370	Прямий доступ до внутрішнього стану в тестах	Тести виставляють user.calculation_count = N напругу, минаючи метод add_to_history(). Після рефакторингу calculation_count стане @property — такий запис викликати помилку. Рекомендація: використовувати add_to_history() у тестах замість прямого присвоєння.	Низький
8	test_calculator.py, рядки 167, 248, 339	Дублювання фікстури у трьох класах	Fixture calc = ProjectCalculator() оголошено окремо в трьох тест-класах. Після видалення ProjectCalculator вони всі стануть непотрібними. Рекомендація: замінити fixture на прямий виклик calculate_cost() без створення об'єкта.	Низький

3. План рефакторингу

Виправлення розташовані за пріоритетом — від найважливішого до найменш критичного:

- Розбити ProjectCalculator: calculate_cost() і generate_report() — окремі функції, клас видалити.

- У класі User прибрати calculation_count як поле; зробити його @property, що повертає len(history).
- Словники-записи {dict} замінити на dataclass CalculationRecord з названими полями.
- Усі числові константи (тарифи, множники, ліміт) винести на рівень модуля.
- for-цикл з накопиченням замінити на sum().
- При невідомій платформі збирати всі помилки одразу, а не зупинятись на першій.
- У тестах прибрати пряме присвоєння user.calculation_count = N — замінити на add_to_history().

Після кожного кроку запускається pytest: тести мають залишатись зеленими.

<https://github.com/yarikknure/bse-lr3-Mohyla/pull/1>

4. Виконаний рефакторинг

4.1. Розбиття класу ProjectCalculator

БУЛО:

```
class ProjectCalculator:
    COMPLEXITY_FACTORS = {1: 1.0, 2: 1.5, 3: 2.5}

    def __init__(self):
        self.platform_rates = {'Webflow': 800, 'Bubble': 1200, ...}

    def calculate_cost(self, platforms_list, complexity): ...
    def generate_report(self, user): ...
```

СТАЛО:

```
def calculate_cost(platforms: list[str], complexity: int) -> float: ...
def generate_report(user: User) -> str: ...
def run_calculation(user: User, platforms: list[str], complexity: int) -> None:
    ...
```

Клас ProjectCalculator видалено. Кожну функцію тепер можна тестувати окремо, не створюючи об'єкт. Цикломатична складність calculate_cost знизилась з 8 до 3.

4.2. Лічильник calculation_count як @property

БУЛО (рядки 38–39, 57–58):

```
self.calculation_history: list = []
self.calculation_count: int = 0

def add_to_history(self, record: dict) -> None:
    self.calculation_history.append(record)
    self.calculation_count += 1 # ручне оновлення
```

СТАЛО:

```
@dataclass
class User:
    username: str
    is_registered: bool = False
    history: list[CalculationRecord] = field(default_factory=list)
```

```

@property
def calculation_count(self) -> int:
    return len(self.history)    # одне джерело правди

def add_to_history(self, record: CalculationRecord) -> None:
    self.history.append(record)

```

Тепер `calculation_count` неможливо випадково збити, бо воно завжди береться з довжини списку.

4.3. Заміна dict на dataclass CalculationRecord

БУЛО: записи зберігались як звичайні словники. При помилці в назві ключа Python не давав жодного попередження:

```

record = {
    'platforms': ['Webflow'],
    'complexity': 1,
    'total_cost': 800.0,
    'timestamp': '2026-06-03',
}

```

СТАЛО:

```

@dataclass
class CalculationRecord:
    platforms: list[str]
    complexity: int
    total_cost: float
    timestamp: str

```

Тепер IDE підказує поля, а опечатки в назвах відловлюються одразу.

4.4. Константи на рівні модуля

БУЛО: тарифи жили в `__init__` об'єкта, множники — в атрибуті класу, мітки — всередині `generate_report`. Одні і ті самі дані в трьох різних місцях.

СТАЛО:

```

from enum import IntEnum
from typing import Final

class Complexity(IntEnum):
    LOW = 1; MEDIUM = 2; HIGH = 3

PLATFORM_RATES: Final = {'Webflow': 800, 'Bubble': 1200, 'Glide': 600, 'Make': 500}
COMPLEXITY_FACTORS: Final = {Complexity.LOW: 1.0, Complexity.MEDIUM: 1.5, Complexity.HIGH: 2.5}
COMPLEXITY_LABELS: Final = {Complexity.LOW: 'Низька', Complexity.MEDIUM: 'Середня', Complexity.HIGH: 'Висока'}

GUEST_LIMIT: Final[int] = 3
REPORT_WIDTH: Final[int] = 50

```

Всі налаштування тепер в одному місці. Змінити тариф — один рядок замість пошуку по всьому файлу.

4.5. sum() та збір усіх невідомих платформ

БУЛО:

```
base_cost = 0
for p in platforms_list:
    if p not in self.platform_rates:
        raise ValueError(f'Невідома платформа: {p}.') # зупиняється на першій
    base_cost += self.platform_rates[p]
```

СТАЛО:

```
unknown = [p for p in platforms if p not in PLATFORM_RATES]
if unknown:
    raise ValueError(f'Невідома платформа: {unknown}.')
base_cost = sum(PLATFORM_RATES[p] for p in platforms)
```

sum() коротший і зрозуміліший. Помилку тепер видно одразу для всіх невідомих платформ.

4.6. Виправлення тестів

БУЛО: тести виставляли calculation_count напряду (рядки 80, 88, 96, 104, 112, 370):

```
user = User('guest')
user.calculation_count = 3 # пряме присвоєння
assert user.can_calculate() is False
```

СТАЛО: стан формується через публічний метод:

```
user = User('guest')
rec = CalculationRecord(['Webflow'], 1, 800.0, '2026-06-03')
for _ in range(GUEST_LIMIT):
    user.add_to_history(rec)
assert user.can_calculate() is False
```

Тепер тести перевіряють поведінку, а не залазять у внутрішні поля класу.

5. Результати тестування

Після рефакторингу запущено повний набір тестів:

```
$ pytest test_calculator.py -v

test_calculator.py::TestUserInit::test_guest_default_attributes PASSED
test_calculator.py::TestUserInit::test_registered_user_flag_is_true PASSED
test_calculator.py::TestUserInit::test_empty_username_raises PASSED
test_calculator.py::TestUserInit::test_non_string_username_raises PASSED
test_calculator.py::TestUserCanCalculate::test_guest_count_0_can_calculate
PASSED
test_calculator.py::TestUserCanCalculate::test_guest_count_1_can_calculate
PASSED
test_calculator.py::TestUserCanCalculate::test_guest_count_2_can_calculate
PASSED
test_calculator.py::TestUserCanCalculate::test_guest_count_3_cannot_calculate
PASSED
test_calculator.py::TestUserCanCalculate::test_guest_count_4_cannot_calculate
PASSED
test_calculator.py::TestUserCanCalculate::test_registered_any_count_can_calcula
te PASSED
... (всього 39 тестів)

===== 39 passed in 0.08s
=====
```

Усі 39 тестів пройшли. Логіка не змінилась — лише адаптовано звернення до API.

6. Порівняння метрик до і після рефакторингу

Таблиця 2 — Метрики якості коду

Метрика	До рефакторингу	Після рефакторингу
Циклом. складність (calculate_cost)	8	3
Кількість класів	2 (User, ProjectCalculator)	1 (User) + dataclass CalculationRecord
Попередження Ruff	6	0
Тести (пройшло / всього)	39 / 39	39 / 39
Ручна синхронізація calculation_count	Є	Немає (@property)

Цикломатична складність розрахована інструментом radon, попередження — ruff check.

7. Висновки

Під час роботи проведено перегляд коду з лабораторної роботи № 3. Знайдено 8 проблем: клас із зайвими обов'язками, дубльований стан, магічні числа, незручний цикл, прямий доступ до полів у тестах та ін.

Виконано рефакторинг: ProjectCalculator розбито на окремі функції, calculation_count замінено на @property, dict-записи — на dataclass, константи зібрано в одне місце. Після рефакторингу всі 39 тестів проходять.

Найцікавіше — те, що проблеми в тестах (прямий запис у calculation_count) вказували на проблему в самому дизайні класу. виправивши дизайн, тести самі стали коректнішими.

Перелік посилань

- Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley.
- Martin, R. C. (2008). Clean Code. Prentice Hall.
- Google Engineering Practices: Code Review. URL: <https://google.github.io/eng-practices/review/reviewer/>
- Radon — Python Code Metrics. URL: <https://radon.readthedocs.io/>
- Ruff Documentation. URL: <https://docs.astral.sh/ruff/>
- Репозитій ЛР 3: <https://github.com/yarikknure/bse-lr3-Mohyla>